

UCS420: Cognitive Computing

Assignment 7 — NLP-I

Open UCS420_LA9_Starter.ipynb in Colab (https://colab.research.google.com/drive/1Xr9A49-kiRxA_HYf4v03XZsy3eBPrgcv?authuser=1) and enter your **roll number**. It generates 12–16 IT-support tickets, and prints a **fingerprint** (8 characters). All your answers must use this dataset. **Do not change generate_dataset() or process_ticket().**

1. Preprocessing - Run process_ticket() on every ticket. Create one table containing, for each ticket:

- number of sentences
- number of tokens
- number of punctuation tokens
- number of stopwords
- number of unique tokens
- number of final tokens

Add a total row for the count-based columns. For one selected ticket, display the actual tokens, punctuation tokens, stopwords and final tokens.

2. Regex extraction and replacement

Using the raw tickets and Python re:

- use re.findall() to extract:
 - words longer than 5 letters
 - numbers, keeping decimals and version numbers such as 95.5 and 2.0.1 as single matches
 - capitalized words
 - words beginning with a vowel
- use re.sub() to replace:
 - email addresses → <EMAIL>
 - URLs → <URL>
 - phone numbers → <PHONE>

Your patterns must also work correctly on the provided TEST_LINES.

3. Custom tokenization

Implement:

my_tokenize(text)

using **one RegexpTokenizer pattern**.

The pattern must preserve:

- contractions: isn't
- hyphenated words: Wi-Fi, state-of-the-art
- decimals: 95.5
- versions: 2.0.1
- email addresses
- URLs
- phone numbers

Run it on TEST_LINES and your tickets.

Compare its output with word_tokenize(). Identify **at least three differences** and explain them.

4. Stemming and lemmatization

On the **unique final tokens**, compare:

- Porter Stemmer
- Lancaster Stemmer
- Regexp Stemmer
- WordNet Lemmatizer with POS information from nltk.pos_tag()

For the Regexp Stemmer, choose a suffix based on **at least two words in your own dataset** and justify your choice.

Create a table containing only the words for which the outputs differ.

5. Corpus statistics

For each of the following:

- **S1:** all tokens
- **S2:** tokens without punctuation
- **S3:** final tokens
- **S4:** S3 after Porter stemming

calculate:

- total tokens
- unique tokens
- TTR

Also:

1. Print the top 10 words of S1 and S3.
2. Plot $\log(\text{rank})$ vs $\log(\text{frequency})$ for S3.
3. Print the top 5 bigrams of S1 and S3.

6. Interpretation

Answer briefly:

(a) Why does TTR change from S1 to S4 in your dataset?

(b) Give one example where `word_tokenize()` and your tokenizer differ. Explain the difference.

(c) Give one example where stemming produces an undesirable result.

(d) Give one example where POS-aware lemmatization preserves meaning better than stemming.